

מס' שאלון - 532 21
ביוני 2021

מס' מועד 57

סמסטר 2021ב

20905 / 4

שאלון בחינת גמר
20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה 9 עמודים

מבנה הבחינה:

- * בבחינה 3 שאלות. עליכם לענות על כל השאלות.
- * משקל השאלות מפורט בגוף השאלון.
- * מבחן בית דרך מערכת הבחינות.

שימו לב:

1. את תשובותיכם יש לרשום על דפים בכתב יד ברור לסרוק וליצור קובץ PDF יחיד ולהעלותו בסיום הבחינה על פי ההנחיות שנשלחו אליכם.
2. כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:

1. ספר הקורס "Essentials of Programming Languages".
2. מדריך הלמידה "שפות תכנות".

בהצלחה !!!

שאלה 1 (30 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

בשאלה זו נרצה להרחיב את השפה, כך שיתאפשר לעבוד עם tuples (בדומה לקיים בשפת פייתון). tuple היא בעצם n-יה סדורה, המאפשרת להחזיק יחד מספר ערכים מסוגים שונים, וכן ניתן לפרק מתוך ה-tuple חלק מהמרכיבים שלו על פי בחירתנו לתוך משתנים.

נרצה להרחיב את שפת "וישגור" עם השינויים הבאים:

- ביטוי חדש המחזיר tuple
- עדכון התחביר ואופן החישוב של ביטוי let כך שידע לעבוד מול tuple, ולאפשר הגדרת מספר משתנים זמניים שערכם מתקבל מחילוף מרכיבים מתוך tuple

להלן הדקדוקים המגדירים את השינויים:

$Expression ::= < \{ Expression \}^{+()}>$

tuple-exp (exps)

$Expression ::= let\ temps = Expression\ in\ Expression$

$temps ::= identifier \mid [\{ _identifier \}^{+()}]$

let-exp (tmps, exp1, body)

הסבר על המרכיבים של הביטויים החדשים:

- ביטוי tuple-exp נכתב בסוגריים משולשים הכוללים בתוכם אוסף של ביטויים (לפחות אחד) המופרדים בפסיקים. הביטוי יחזיר tuple המייצג את אוסף תוצאות רשימת הביטויים.
- לביטוי let החדש, מבנה דומה לביטוי let הקיים כבר בהגדרות השפה, פרט לכך שבמקום הגדרת משתנה זמני בודד כפי שהיה עד כה, נוכל לרשום במקום זאת מרכיב המוגדר ע"י temps. מרכיב temps מאפשר לבחור להציב או שם משתנה בודד כפי שהיה עד כה, או שמות משתנים שיקבלו ערכים המחולצים מתוך tuple. שמות המשתנים ניתנים בתוך סוגריים מרובעות, ובתוכן רשימת שמות משתנים מופרדים בפסיקים עם מבנה תואם ל-tuple ממנו מחלצים ערכים לתוך המשתנים, אם ברצוננו לא לחלץ מרכיבים מסויימים מתוך tuple אזי במקום שם משתנה ייכתב _ (מקף תחתון) וכך נוכל להתעלם מחלק ממרכיבי ה-tuple ולחלץ רק את חלקם. בתוך הסוגריים המרובעות חייב להופיע לפחות שם אחד של משתנה, לא ניתן שכל השמות יהיו _ (מקף תחתון)

דוגמאות להמחשת אופן השימוש במרכיבים החדשים:

דוגמה 1:

```
> (run
  "let t1=<20,proc (x) -(x,1), 50>
  in
    let [_p,_] = t1
    in (p 200)")
(num-val 199)
> |
```

הסבר דוגמה 1:

בדוגמה 1, ב-let החיצוני מוגדר משתנה בשם t1 (כמו בתחביר המקורי) וערכו הוא tuple המכיל 3 מרכיבים, הראשון המספר 20, השני פרוצדורה, והשלישי המספר 50. בביטוי ה-let הפנימי, מוגדר משתנה p וערכו הוא המרכיב השני מתוך ה-tuple שהוגדר קודם (t1). מאחר ואנו לא מעוניינים לחלץ מתוך ה-tuple את המרכיב הראשון והשלישי שלו, נרשמו במקומות אלה _ (מקפים תחתונים). ולכן, ה-let הפנימי הגדיר משתנה p שערכו הוא פרוצדורה.

ב-body של ה-let הפנימי מופעלת הפרוצדורה ש-p מייצג על הערך 200, ופרוצדורה זו מחזירה 199.

שימו לב, ה-let החיצוני אינו פעולת חילוץ מ-tuple (אין שם שימוש בסוגריים מרובעות), אלא הגדרת משתנה בשם t1 שערכו הוא כל ה-tuple

דוגמה 2:

```
> (run
  "let [_a,_b,_] = <15,25,35,45,55,65>
  in -(b,a)")
(num-val 30)
>
```

הסבר דוגמה 2:

בדוגמה 2, ביטוי ה-let מגדיר 2 משתנים זמניים, a ו-b שאת ערכם הם מקבלים מחילוץ המרכיבים השני והחמישי מתוך ה-tuple <15,25,35,45,55,65>, ולכן הערך של משתנה a הוא (num-val 25) וערכו של משתנה b הוא (num-val 55), ולכן תוצאת ה-body היא 25-55, וערכה של כל התוכנית הוא (num-val 30)

שימו לב, בחילוץ איברים מ-tuple, נדרש שצד ימין יהיה ביטוי שתוצאתו מסוג tuple, וצד שמאל יהיה בעל מספר מרכיבים תואם לזה ב-tuple.

יש לשים לב ולבדוק מקרים של שימוש לא תקין בתחביר (כגון: רשימות שלא מכילות לפחות איבר אחד, חילוץ מתוך ערך שאינו tuple, חילוץ במבנה שאינו תואם למבנה ה-tuple וכדומה) במקרים של שימוש לא תקין, יש להדפיס הודעות שגיאה מתאימות.

ממשו והטמיעו את השינויים הדרושים בתוך שפת "וישגור" (שפת PROC). הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בקבצי המפרש, ומהם השינויים והתוספות.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים.
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (' } ' או { })
- 7) ניתן להוסיף במקרה הצורך לדקדוק השפה מרכיבי-עזר נוספים (בדומה ל-temps המוגדר בשאלה)

שאלה 2 (45 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמו" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם switch-exp

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

```
Expression ::= switch Expression {  
    {Type identifier when (Expression) => Expression}(*)  
    default => Expression  
}
```

Type ::= number | boolean | function

switch-exp (e1 typs ids bools exps)

שימו לב, Type המוגדר בשאלה זו, אינו קשור כלל ל-Type שפגשנו בפרק 7.

הסבר על מרכיבי הביטוי ואופן פעולתו:

ביטוי switch-exp מורכב מהמרכיבים הבאים:

- e1 – הוא ביטוי שאת תוצאתו רוצים לבדוק.
- typs – רשימה של טיפוסים מסוג Type.
- ids – רשימה של identifiers.
- bools – רשימת ביטויים שתוצאת כל אחד מהם בוליאנית.
- exps – רשימת ביטויים כלשהם.
- defexp – ביטוי כלשהו.

ביטוי switch-exp מחשב את ערכו של e1 ולאחר מכן מחפש באינדקסים תואמים מתחילת הרשימות typs, ids, bools, exps, האם קיימת התאמה בין טיפוס הערך של e1 לטיפוס באינדקס הנבדק (למשל אם ערכו של הביטוי הוא מסוג num-val ובשורה הטיפוס הוא number אז יש התאמה, כנ"ל עבור bool-val ו-boolean, ועבור proc-val ו-function), אם אין התאמה, יש לעבור לאינדקס הבא ברשימות. אם נמצאה התאמה, יש לחשב את ערכו של הביטוי הנמצא באינדקס הנוכחי ברשימה bools, באמצעות סביבה הבנויה מהרחבה של הסביבה הנוכחית, עם משתנה חדש בשם כפי המופיע ברשימת ids באינדקס הנוכחי, וכריכתו לערך של הביטוי e1. אם חישוב הביטוי הניב תשובה בוליאנית של false יש להמשיך הלאה לאינדקס הבא, ואם חישוב הביטוי הניב תשובה בוליאנית של true, יש להפסיק את כל החישוב של ביטוי ה-switch ולהחזיר את ערכו של הביטוי הנמצא באינדקס הנוכחי ברשימה exps.

אם לא נותרו איברים ברשימות (כי לא נמצא באף אינדקס ברשימות איברים המקיימים את התנאים לעיל) במקרה זה, ביטוי switch-exp יחזיר את ערכו של הביטוי defexp להלן דוגמאות להמחשת השימוש בביטוי switch-exp :

דוגמה 1:

```
> (run
  "let x=15
    in
      let y= switch proc (b) -(b,4)
        {
          boolean i when (if i then zero? (x) else zero?(0) ) => 90,
          number a when ( zero?(x)) => 100,
          function f when ( zero? (-(x,15))) => (f 300),
          number i when ( zero? (-(i,11))) => 200
          default => 400
        }
      in
        y")
(num-val 296)
> |
```

בדוגמה 1, ביטוי switch מקבל לבדיקה ביטוי שערכו פרוצדורה, ביטוי switch בודק את השורה הראשונה, ושם כתוב boolean ואין כאן התאמה לסוג הביטוי הנבדק (כי הוא פרוצדורה ולא בוליאני), ולכן עוברים לבדוק את השורה הבאה, שאף היא אינה מתאימה (כתוב שם number). בשורה השלישית נמצאת המילה function ולכן שורה זו תואמת לטיפוס הנבדק, ולכן בודקים כעת האם הביטוי שכתוב לאחר המילה when ערכו הוא true, ואכן, הביטוי (-(x,15)) zero? הוא true כי ערכו של x הוא 15, לכן מצאנו שורה תואמת לכל התנאים, וערכו של כל ביטוי ה-switch יהיה כערכו של הביטוי (f 300) המחזיר 296 ולכן ערכו של y המוחזר הוא 296.

דוגמה 2:

```
> (run
  "let x=15
    in
      let y= switch -(x,4)
        {
          boolean i when (if i then zero? (x) else zero?(0) ) => 90,
          number a when ( zero?(a)) => 100,
          number i when ( zero? (-(i,11))) => 200,
          function f when ( zero? (x)) => 300
          default => 400
        }
      in
        y")
(num-val 200)
>
```

בדוגמה 2, ערכו של הביטוי הנבדק ע"י switch הוא מסוג num-val ולכן רק שורות מסוג number באות בחשבון, השורה הראשונה מסוג number אינה מתאימה משום שערכו של הביטוי שלאחר המילה when הוא false (כי ערכו של a הוא 11 ולא 0) ולכן שורה זו אינה מתאימה. השורה השלישית היא גם מסוג number ובה ערכו של הביטוי לאחר המילה when הוא true, כי ערכו של i הוא 11. ולכן מצאנו שורה מבוקשת וערכו של כל ביטוי ה-switch יהיה 200, ולכן ערכה של כל התוכנית הוא כערכו של y שקיבל 200.

דוגמה 3:

```
> (run
  "let x=15
  in
    let y= switch proc (b) -(b,4)
      {
        boolean i when (if i then zero? (x) else zero?(0) ) => 90,
        number a when ( zero?(x)) => 100,
        function f when ( zero? (x)) => (f 300),
        number i when ( zero? (-(i,11))) => 200
        default => 400
      }
    in
      y")
(num-val 400)
>
```

בדוגמה 3, אף אחת מהשורות אינה מתאימה לביטוי הנבדק (או שאינה מהטיפוס המתאים או שהתנאי הבוליאני הוא false) ולכן מוחזר ערכו של הביטוי משורת ה-default שהוא 400

דוגמה 4:

```
> (run
  "let x=15
  in
    let y= switch proc (b) -(b,4)
      {
        default => 400
      }
    in
      y")
  interp.scm:115:20: Error : switch cases is missing
>
```

בדוגמה 4, נרשמה רק שורת ה-default ועל פי הדקדוק הנתון בשאלה נדרשת לפחות שורה אחת של תנאים בנוסף ל-default, ועל כן, הביטוי אינו נכון תחבירית, ולכן התקבלה הודעת שגיאה יזומה על כך שחסרים מקרי בדיקה.

ממשו והטמיעו את ביטוי switch-exp בתוך שפת **"וירמוז"** (שפת IMPLICIT-REFS).
הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

יש לשים לב ולבדוק מקרים של שימוש לא תקין בתחביר.
במקרים של שימוש לא תקין, יש להדפיס הודעות שגיאה מתאימות.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (}) או פתוח ({ }

שאלה 3 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
proc (x:?)  
  if (x proc (y:?) zero?(-(y,3)))  
  then 80  
  else 90
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !